HƯỚNG DẪN KIỂM TRA & NGHIỆM THU ĐỒ ÁN BTL

HỌC PHẦN: KỸ THUẬT LẬP TRÌNH AN TOÀN (CSE703093 / CSE703153)

Giảng viên hướng dẫn: ThS. Vũ Quang Dũng — Đại học Phenikaa

Nhóm 11: Chủ đề 4 — Dự án Fuzzing (FZP1 Binary Parser)

. . . . . .

1. TỔNG QUAN DỰ ÁN

- **Đề tài:** Phân tích Cú pháp Gói tin Nhị phân FZP1 & Đo lường Đối sánh 3 Trường phái Fuzzing (Black-box vs Greybox vs White-box Concolic SMT Z3).
- **Thành viên nhóm:** Lưu Đức Hiệp (50%) & Hà Nguyễn Trúc Linh (50%) — *[Mã sinh viên và Lớp bảo mật theo quy định]*.
- **Cấu trúc gói nộp:**
- `specification/Nhom11_FZP1_Fuzzing_Spec.xlsx`: Tệp đặc tả 10 sheet chuẩn SSOT theo yêu cầu môn học.
- `source/`: Mã nguồn C bản lỗi (`fzp1_parser_v0.c`) và bản vá CERT C (`fzp1_parser_v1.c`).
- `report/BaoCao_BTL_Nhom11.docx` (và bản sao tại thư mục gốc): Báo cáo Word chính thức chuẩn định dạng đồ án Phenikaa.
- `report/BaoCao_BTL_Nhom11.pdf`: Bản PDF tương ứng lưu trữ theo chuẩn môn học.
- `results/`: Dữ liệu thô JSON của 30 lượt chạy benchmark, biểu đồ và nhật ký kiểm định.

. . . . . .

2. CÁC LỆNH KIỂM TRA VÀ TÁI LẬP KẾT QUẢ (REPRODUCIBILITY)

Bước 1: Biên dịch mã nguồn C

Môi trường Linux / WSL / Clang (AddressSanitizer):

```
gcc -fsanitize=address -O0 -g source/fzp1_parser_v0.c -o source/fzp1_parser_v0_asan
gcc -fsanitize=address -O0 -g source/fzp1_parser_v1.c -o source/fzp1_parser_v1_asan
```

Môi trường Windows (MinGW-w64 UCRT64 với Stack Canary):

```
gcc -Wall -Wextra -O0 -g -fstack-protector-all source/fzp1_parser_v0.c -o
source/fzp1_parser_v0.exe
gcc -Wall -Wextra -O0 -g -fstack-protector-all source/fzp1_parser_v1.c -o
source/fzp1_parser_v1.exe
```

Bước 2: Chạy kiểm thử tự động toàn diện (Pytest 14/14)

```
pytest tests/ -v
# Kết quả mong đợi: 14 passed trong < 2.5 giây (10 unit test cases FZP1 + 4 test cases
kiểm chứng tiến hóa đa phiên bản CWE-121, CWE-415, CWE-416 và bản vá CERT C)
```

Bước 2b: Kiểm tra tệp Đặc tả SSOT trên Công cụ `SecLabFramework.exe` (Tab Ch6)

1. Mở công cụ `TAI_LIEU_VA_YEU_CAU/04_Cong_Cu_Giang_Vien/SecLabFramework.exe`.
2. Chọn Tab **Ch6: Spec -> Kripke -> Code -> CWE/CVE**.
3. Chọn tệp: `specification/Nhom11_FZP1_Fuzzing_Spec.xlsx`.
4. Nhấn **Kiểm chứng Kripke & Thuộc tính CTL** (Đồ thị Kripke 4 trạng thái, kiểm tra deadlock-free và cảnh báo xung đột).

5. Nhấn **Sinh mã C/Python từ đặc tả** để quan sát các khuôn mẫu hàm kiểm toán an toàn bộ nhớ.

Bước 2c: Phân tích Chuỗi Truy vết Gói tin Mạng IoT (PCAP -> SSOT)

```
python scripts/analyze_pcap_to_spec.py
# Trích xuất gói tin nhị phân từ network_capture/iot_sensor_stream.pcap
# Xuất nhật ký hex dump và ma trận ánh xạ đặc tả tại results/logs/pcap_traffic_log.txt
```

Bước 3: Kiểm chứng 3 Lỗ hổng Bộ nhớ Thực tế và Bản vá bằng tệp tấn công

1. Lỗ hổng 1 — CWE-121 (Stack-based Buffer Overflow):

```
# Bản v0 sập tiến trình do ghi đè Stack Canary (STATUS_STACK_BUFFER_OVERRUN)
.\source\fzp1_parser_v0.exe .\crashes\whitebox_s0_crash_0.bin
# Bản v1 từ chối an toàn theo chuẩn CERT STR31-C (Exit Code: 2)
.\source\fzp1_parser_v1.exe .\crashes\whitebox_s0_crash_0.bin
```

2. Lỗ hổng 2 — CWE-415 (Double Free):

```
# Bản v0 sập tiến trình do giải phóng Heap lặp lại khi gặp lệnh abort 0xFF
(STATUS_HEAP_CORRUPTION 0xC0000374)
.\source\fzp1_parser_v0.exe .\crashes\crash_cwe415.bin
# Bản v1 xử lý an toàn, reset con trỏ NULL theo chuẩn CERT MEM30-C & MEM31-C (Exit Code:
2)
.\source\fzp1_parser_v1.exe .\crashes\crash_cwe415.bin
```

3. Lỗ hổng 3 — CWE-416 (Use-After-Free):

```
# Bản v0 sập tiến trình do gọi hàm callback qua con trỏ lơ lửng sau khi giải phóng kênh
stream 0xFE
.\source\fzp1_parser_v0.exe .\crashes\crash_cwe416.bin
# Bản v1 xử lý an toàn theo chuẩn CERT MEM30-C (Exit Code: 2)
.\source\fzp1_parser_v1.exe .\crashes\crash_cwe416.bin
```

Bước 3b: Chạy Chu trình Fuzzing Tiến hóa Đa Giai đoạn (Crash -> Vá -> Đổi Seed -> 0 Crash)

```
python scripts/run_iterative_fuzz_patch_lifecycle.py
# Chạy chu trình kiểm thử tiến hóa qua 4 mốc: v0 (90 crashes) -> v0.1 (58 crashes) -> v0.2
(22 crashes) -> v1 (0 crash)
# Tệp crash mẫu của 3 mốc: crashes/iter1_cwe121_crash.bin, iter2_cwe415_crash.bin,
iter3_cwe416_crash.bin
# Nhật ký: results/logs/iterative_crash_evolution_log.txt | Dữ liệu:
results/data_raw/iterative_crash_evolution.json
```

Bước 4: Chạy lại Benchmark 30 lượt thử nghiệm & Sinh Biểu đồ

```
python scripts/run_fzp1_benchmark.py
python scripts/aggregate_results.py
python scripts/generate_charts.py
# Dữ liệu được cập nhật tại results/data_raw/ và biểu đồ tại results/figures/
```

Bước 5: Chạy công cụ thẩm định 8 khâu toàn diện dự án

```
python scripts/verify_project.py
# Kết quả mong đợi: TẤT CẢ 8 KHÂU ĐẠT CHUẨN (PASS 100%)
```

. . . . . .

3. TÓM TẮT KẾT QUẢ ĐẠT ĐƯỢC

- **Tỷ lệ tìm thấy lỗi qua 30 trials:**
- Black-box (Ngẫu nhiên mù quáng): **0% (0/30)** — Rào cản toán học P = 1/2^{40} ≈ 9.09 × 10^{-13}.
- Greybox (Biết Magic header): **46.67% (14/30)** — Trung vị 2.16 giây (160.5 vòng lặp).
- White-box (Z3 SMT Solver): **100% (30/30)** — Thời gian giải Z3 trung bình **6.98 ms** (trung vị time-to-crash 0.21 giây), tìm ra crash ngay trong vòng lặp đầu tiên.

- **Chuẩn hóa bản vá:** Tuân thủ 100% **CERT STR31-C** (Stack bound check), **CERT MEM30-C / MEM31-C** (Dangling pointer / UAF prevention & Idempotent NULL heap cleanup).